

Kapitel 15. Ärvning

Innehållsförteckning

[Mål med ärvning](#)[Konkret exempel](#)[Överlagring av metoder](#)[Typer av ärvning](#)[Referera till basklasser](#)

Detta kapitel behandlar de olika arvsmekanismerna som är tillgängliga i C++. Ärvning kan användas till många olika ändamål i C++.

Mål med ärvning

Ärvning är en av de grundläggande byggstenarna inom objektorienterad programmering och även inom C++. Att programmera i C++ utan att kunna använda sig av styrkan som ärvning är som att köra en Ferrari på endast första växeln på halv gas. Det är inte detta kompendiums uppgift att försöka förklara teorin och begreppen bakom ärvning. Vi skall se på några av fördelarna med ärvning och några av de olika modellerna för ärvning.

Några av de grundläggande målen med ärvning av klasser är att kunna skapa nya klasser som är antingen *specialiserade* eller *utvidgade* versioner av basklassen. Med specialiserad avses att man begränsar den nya klassens funktionalitet till ett mindre område än den ursprungliga klassens. Om man t.ex. har en klass som definierar en *atom* kan man ärva *subklasser* för olika typer av grundämnen. Man specialiserar då de nya klasserna att fungera endast som ett visst ämne. Det andra stora målet är att skapa utvidgade versioner av basklassen som har ökad funktionalitet för att hantera andra uppgifter. Man kan t.ex. ärva en *register*-klass från en *lista*. Man ökar då på den funktionalitet som finns i ursprungsklassen och lägger till metoder för att hantera t.ex. olika typer av poster i registret.

Relationer mellan objekt

Det finns olika teoretiska förhållanden mellan klasser i ett program. Det förhållande som ärvning hanterar brukar kalla *is-a* -förhållanden, vilket betyder att den ärvda klassen är av samma typ som ursprungsklassen. Om man t.ex. har en basklass `Frukt` och ur denna ärver de nya klasserna `Banan` och `Appelsin` kan man säga att de nya klasserna *är* frukter. En `Banan` är även en `Frukt` och har därför samma medlemmar och metoder som denna, som t.ex. vikt och pris.

Andra förhållanden är *has-a* som används för att representera nästlade klasser (containment). T.ex. en klass `Lunch` innehåller kanske en `Banan`, men det är inte vettigt att ärva klassen `Lunch` från `Banan` endast av denna orsak. Däremot är det vettigt att ha en medlem `Banan` i klassen `Lunch`. Förhållandet *has-a* kan dock sällan uppfyllas genom ärvning.

Ett tredje förhållande mellan klasser kallas *is-implemented-as-a*, och betyder att en klass är implementerad genom att använda en viss klass. Man kan t.ex. implementera en stack genom att använda en dynamisk vektor, men en stack är inte en vektor trots att den är implementerad med hjälp av en sådan. I sådana fall kan ärvning användas, men det är inte alltid vettigt. Bättre är även där att

använda sig av nästlade klasser och ge klassen `Stack` en privat medlem `Vector`.

Det sista förhållandet är en form av *uses-a*, där en viss klass använder sig av en annan klass för att utföra någonting. Inte heller detta förhållande uttrycks vettigt med hjälp av ärvning. Det enda förhållandet mellan objekt som alltså bör uttryckas via ärvning är *is-a*. Det kan ibland vara svårt att känna igen dessa förhållanden mellan objekt, men det är essentiellt för att undvika konstgjorda ärvningar eller nästlade klasser.

Återanvändning av kod

En av de fundamentala hörnstenarna inom det man vill åstadkomma med OO är *återanvändning av kod*. Varför uppfinna hjulet om och om igen om man kan använda kod som redan finns? Utan objektorientering är återanvändning av kod relativt klumpig och begränsar sig till användning av funktioner ur ett bibliotek och cut'n'paste från existerande program. Genom att planera klasser i ett objektorienterat språk så att de är lätta att återanvända löser många problem och spara mycket tid om det görs rätt. Med återanvändbara klasser kan man bygga upp olika *klassbibliotek* vars klasser andra programmerare sedan kan använda eller ärva ifrån.

Det är dock relativt svårt att göra klasser som är enkla att återanvända, men vi skall i detta kapitel gå igenom de olika metoder som finns tillgängliga för ärvning inom C++. Vi skall även titta på några implementationsdetaljer som man bör vara medveten om då man tänker sig att klasserna skall kunna återanvändas på ett vettigt sätt. Det krävs tyvärr en del övning innan man klarar av att göra bra återanvändbara klasser i C++.

Terminologi

Här finns några begrepp som förekommer runt i texten samlade.

- *basklass* är den klass en viss klass ärver ifrån. Den klass kan ha flera basklasser, ifall det är frågan om en arvshierarki där flera led av ärvning förekommer.
- *subklass* är en ärvd klass. Motsatsen till basklass.
- *superklass* är samma som basklass.

[Förutgående](#)

Nästlade klasser

[Hem](#)

[Nästa](#)

Konkret exempel